https://developer.nvidia.com/blog/unified-memory-cuda-beginners/
https://developer.nvidia.com/blog/even-easier-introduction-cuda/

I went over both of these papers. The intro paper has a lot of overlap with my readings from last week. However it goes into unified memory and has some very enlightening changes that have a serious impact on performance.

In my vecAdd and vecAddManaged code, I initialize the arrays on the CPU. The issue is when I use cudaMallocManaged, the pages of memory can be either host or device resident. If it is on the GPU when we initialize then we will get page faults and the memory needs to be migrated over. Even worse, once we launch the kernel, the memory is now CPU resident and the GPU will get page faults and the memory will again need to be migrated over.

I can inspect this behavior with nsys profiling tools.

A solution to this problem is cudaMemPrefectAsync. This allows us to move the data to the GPU before the kernel needs it.

The second paper talks about performance and behavior of cudaMallocManaged between kepler and pascal GPUs.

On the Kepler GPUs it describes some of the issues I outlined above. Specifically when you use managed memory, the memory is initially resident on the GPU. So my vecAddManaged code is actually structured/designed very poorly, or without these considerations. Because the memory is initially GPU resident, then I initialize it with the CPU, I get the page faults and data migration, then I launch the kernel without any prefecting which causes the exact same thing to happen on the GPU.

Another solution outside of prefetching, is to initialize on the GPU.

We then go into using profilers to measure different ways to avoid this migration overhead.

There is an interesting section on the concurrency of accessing the same memory simultaneously from the CPU and GPU. On older devices and cuda versions, this was not possible because page faults didn't exist. On modern devices page faults are possible and so you can have simultaneous access between host and device. However you can still have race conditions that you are responsible for.

This author has a ton of papers and articles that I want to read more of.

https://developer.nvidia.com/blog/how-optimize-data-transfers-cuda-cc/
https://developer.nvidia.com/blog/how-overlap-data-transfers-cuda-cc/

I will look over these but I don't want to get too side tracked.

When talking about optimizing data transfers the first topic is pageable memory transfers vs pinned memory transfers. I'm familiar with pinning memory as I work a lot with DMA/RDMA on infiniband and other types of devices. The process essentially being that we pin the page so it is not moved or swapped out of memory. In the infiniband/rdma world you use libibverbs ibv_reg_mr. This can be a very slow process though, because of the overhead it's usually only used for larger data transfers, as the original staging designs tend to be faster for smaller messages.

In this paper, it discusses the pageable memory transfers. First they will end up creating pinned memory anyways to use as the staging area before transferring the data. The optimization is to just always use pinned memory if you're going to be transferring data. You can do this with cudaMallocHost

The paper even mentions this overhead cost I was talking about before. Due to the cost of using pinned memory, it's best to batch small messages into a single large transfer by packing the messages into a single buffer then sending it.

Another important concern is that using pinned memory should be minimal as it lowers the amount of available memory to the system overall.

The next paper is talking about streams and overlapping data transfers.

A stream is a sequence of operations that execute on the device in the order in which they are issued by the host code.

Operations within a stream execute in the prescribed order, operations on different streams run in their own order.

When no stream is specified the default stream is used (null stream). Operations by default synchronize with the default stream. Meaning no operation on the default stream will begin until all previously issued operations in any stream complete.

They give an example of how you can overlap device and host side computation. Because the kernel launch is asynchronous if you follow it up with a host side call they will overlap. And the final device to host memory copy will only happen after the kernel launch completes.

In order to perform data transfers on non default streams we need to use cudaMemcopyAsync, which takes a stream parameter as an argument. This call is non blocking on the host so control returns to the host thread immediately.

These are non default streams.

All operations on non default streams are non blocking from the host perspective. If you need to synchronize on the host side there are several options cudaDeviceSynchronize, cudaStreamSynchronize, cudaEventSynchronize etc. Depends on the level you want to synchronize.

Now we want to do overlapping kernel and data transfers. So far we have only overlapped kernel with host side code.

In order to achieve this we must:

- Have a device that is capable of concurrent copy and execution. This can be queried from DeviceOverlap field of cudaDeviceProp struct. Or the output of deviceQuery.
- The kernel execution and the data transfer to be overlapped must both occur in different non default streams.
- the host memory involved must be pinned memory.

Now we have two code examples that achieve different types of overlap and have very different performance based off of the device being run on.

You can see how Asynchronous performance changes between version 1 and 2 depending on the device. This comes down to how cuda devices schedule and execute tasks. One device has a single copy engine and a single kernel engine. The other device has two copy engines, one for H→D and another for D→H as well as a single kernel engine.